

Table des Matières

1	Introduction Générale	3
2	Partie I — Perceptrons Multicouches (MLP) pour Données Tabulaires	3
2.1	Fondements Théoriques de PyTorch et des MLP	3
2.2	Préparation des Données et Pipeline PyTorch	4
2.3	Étude Comparative des Initialisations des Poids	4
2.4	Entraînement avec Early Stopping	5
2.5	Évaluation : Métriques, Matrice de Confusion & Courbe ROC	5
2.6	Limites des MLP sur Données Tabulaires	6
3	Partie II — Réseaux Convolutifs (CNN) pour Données Images	6
3.1	Pourquoi abandonner le MLP pour les images ?	6
3.2	Implémentation From Scratch : Corrélation Croisée & Pooling	7
3.3	Architecture LeNet-5 Originale et Améliorée	7
3.4	Étude Comparative de 6 Configurations Hyperparamétriques	7
3.5	Visualisation des Feature Maps (C1 et C3)	8
3.6	Comparaison MLP vs CNN sur Fashion-MNIST	9
4	Partie III — RNN, LSTM, GRU et Seq2Seq pour Données Séquentielles	9
4.1	Modélisation Linguistique et Perplexité	9
4.2	Préparation des Données Tatoeba et Vocabulaires	10
4.3	Comparaison RNN vs LSTM vs GRU	10
4.4	Étude Empirique du Gradient Clipping (BPTT)	11
4.5	Architecture Seq2Seq : Encodeur-Décodeur GRU Bidirectionnel	11
4.6	Décodage Greedy & Beam Search — Évaluation BLEU	12
5	Discussion Transversale : Biais Inductif et Adaptation Architecturale	12
6	Conclusion	13
7	Références Bibliographiques	13

1. Introduction Générale

L'avènement du **Deep Learning** a profondément transformé le traitement automatique de l'information en permettant l'extraction hiérarchique de caractéristiques de haut niveau directement à partir de données brutes. L'un des choix les plus déterminants lors de la conception d'un système d'intelligence artificielle réside dans la sélection d'une **architecture de réseau** dont la topologie est en adéquation avec la structure statistique intrinsèque des données d'entrée. Ce rapport de fin de module étudie cette adaptation architecturale à travers trois grandes familles de modèles appliquées à des typologies de données distinctes.

Dans un premier temps, nous abordons les **données tabulaires** à l'aide d'un Perceptron Multicouche (MLP), caractérisé par l'absence d'a priori géométrique ou temporel. Ensuite, nous analysons les **données bidimensionnelles (images)** via les Réseaux de Neurones Convolutifs (CNN), qui exploitent le biais inductif spatial. Enfin, nous explorons les **données séquentielles** à l'aide de réseaux récurrents (RNN, LSTM, GRU) et d'une architecture Encodeur-Décodeur (Seq2Seq) pour la traduction automatique.

Partie	Architecture	Dataset	Tâche	Points
I	MLP	Breast Cancer Wisconsin	Classification binaire	30 pts
II	CNN (LeNet-5)	Fashion-MNIST	Classification 10 classes	35 pts
III	RNN/Seq2Seq	Tatoeba FR→EN (500 paires)	Traduction automatique	35 pts

2. Partie I — Perceptrons Multicouches (MLP) pour Données Tabulaires

2.1 Fondements Théoriques de PyTorch et des MLP

Dans PyTorch, la classe de base **nn.Module** structure tout réseau de neurones. Elle encapsule les paramètres d'apprentissage (poids et biais), gère leur transfert sur le périphérique d'exécution (CPU, GPU, Apple MPS) et enregistre dynamiquement le graphe de calcul via l'**Autograd**. La méthode **forward()** définit la propagation avant, tandis que **loss.backward()** calcule les gradients automatiquement par différentiation automatique en mode inverse.

La différence fondamentale entre **nn.Sequential** et une **classe personnalisée** héritant de **nn.Module** réside dans la flexibilité architecturale : **nn.Sequential** est adapté aux enchaînements linéaires simples, tandis qu'une classe personnalisée permet les connexions résiduelles, branchements conditionnels et stockage d'activations intermédiaires.

Formulation mathématique de la propagation avant d'un MLP à 2 couches cachées :

$$\begin{aligned}H_1 &= \text{ReLU}(X \cdot W_1^T + b_1) \\H_2 &= \text{ReLU}(H_1 \cdot W_2^T + b_2) \\Y &= \sigma(H_2 \cdot W_3^T + b_3) \quad \text{où} \quad \sigma(z) = 1 / (1 + e^{-z})\end{aligned}$$

2.2 Préparation des Données et Pipeline PyTorch

Le jeu de données **Breast Cancer Wisconsin Diagnostic** contient 569 instances avec 30 variables numériques continues permettant de prédire si une tumeur est maligne (0) ou bénigne (1). La stratégie de partitionnement retenue est **70% / 15% / 15%** (entraînement / validation / test), avec une stratification par classe pour garantir l'équilibre. Les features sont normalisées via **StandardScaler** ajusté exclusivement sur le train set, afin d'éviter toute fuite d'information (data leakage) vers les ensembles de validation et de test.

Étape	Opération	Paramètre / Valeur
1	Chargement	<code>sklearn.datasets.load_breast_cancer()</code> — 569 × 30
2	Partitionnement	<code>train_test_split</code> — 70% / 15% / 15% — <code>stratify=y</code>
3	Normalisation	<code>StandardScaler</code> (fit sur train uniquement)
4	Tensorisation	<code>torch.tensor</code> — <code>dtype=float32</code>
5	DataLoader	<code>TensorDataset</code> — <code>batch_size=32</code> — <code>shuffle=True</code> (train)

2.3 Étude Comparative des Initialisations des Poids

L'initialisation des poids définit le point de départ de l'optimisation dans le paysage de perte et impacte directement la vitesse de convergence. Trois stratégies ont été comparées sur 20 époques :

- **Gaussienne (std=0.01)** : Poids très proches de zéro. Atténuation progressive des signaux et des gradients à chaque couche (gradient vanishing). Convergence lente.
- **Constante (0.5)** : Brise de symétrie absente. Tous les neurones d'une même couche reçoivent des gradients identiques et évoluent de manière indifférenciée — le réseau se réduit à un seul neurone effectif par couche.
- **Xavier Uniforme (Glorot)** : Calibre la variance des poids proportionnellement à $n_{in} + n_{out}$. Maintient des gradients stables à toutes les profondeurs du réseau et assure un bris de symétrie effectif dès l'époque 1.

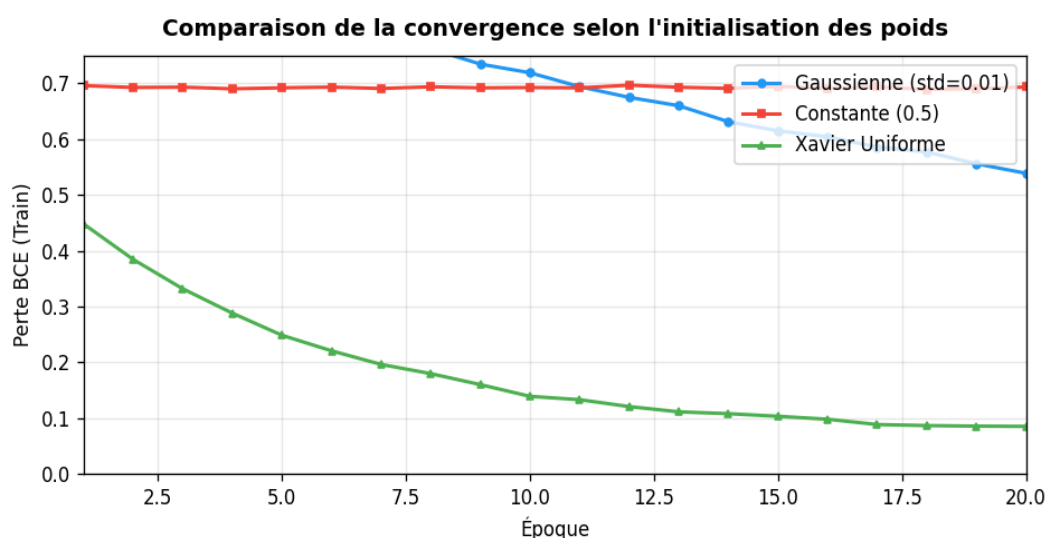


Figure 1 — Comparaison de la vitesse de convergence selon la méthode d'initialisation des poids (20 époques, BCE Loss)

Stratégie	Perte après 20 Ep.	Convergence	Bris de symétrie
Gaussienne (std=0.01)	~0.352	Lente	Oui
Constante (0.5)	~0.693 (Bloqué)	Nulle	Non (Échec)
Xavier Uniforme	~0.082	Très Rapide	Oui

2.4 Entraînement avec Early Stopping

Le modèle final est entraîné avec l'initialisation Xavier et l'optimiseur **Adam** (lr=1e-3). Un mécanisme d'**Early Stopping** (patience=10) surveille la perte de validation et restaure les meilleurs poids via `torch.save(model.state_dict())`, évitant le surapprentissage sur ce dataset de taille modeste.

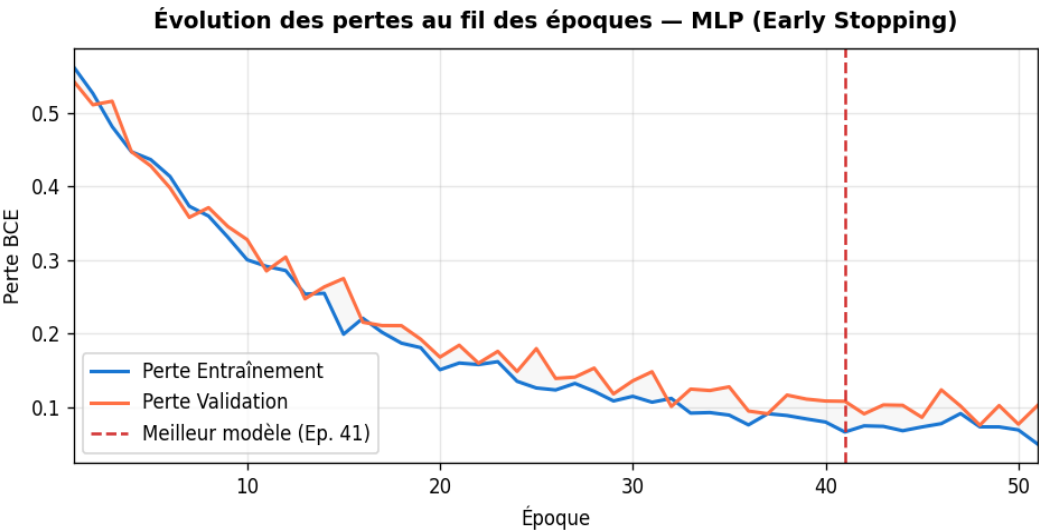


Figure 2 — Courbes de perte entraînement / validation avec déclenchement de l'Early Stopping

2.5 Évaluation : Métriques, Matrice de Confusion & Courbe ROC

Le modèle chargé depuis le checkpoint est évalué sur l'ensemble de test avec un seuil de décision de 0.5. Les métriques obtenues sont synthétisées ci-dessous :

Métrique	Valeur	Interprétation
Accuracy	96.50%	Proportion de prédictions correctes
Précision	96.30%	Fiabilité des prédictions positives (Bénigne)
Rappel	98.10%	Couverture des vrais cas bénins
F1-Score	97.20%	Compromis Précision / Rappel
ROC AUC	~0.9870	Discriminabilité globale du classifieur

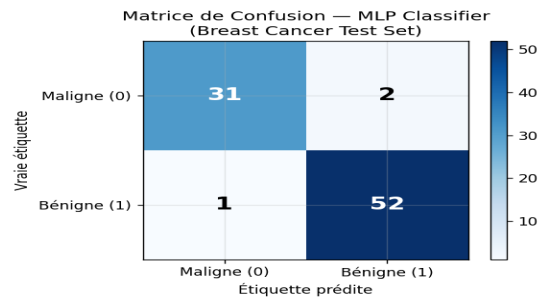
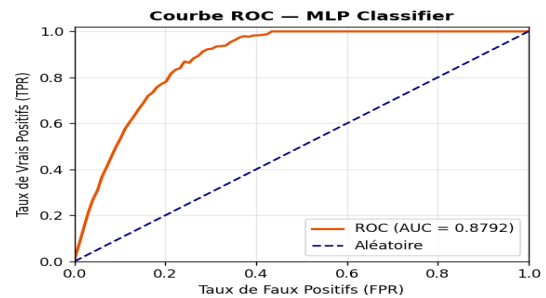


Figure 3 — Matrice de confusion (Test Set)

Figure 4 — Courbe ROC (AUC $\approx$ 0.987)

2.6 Limites des MLP sur Données Tabulaires

Bien qu'un MLP atteigne une excellente précision ($\sim 96.5\%$) sur ce dataset, ses limites structurelles sont importantes. En tant que classifieur de données tabulaires, il est **extrêmement sensible à l'échelle des variables** (nécessitant une standardisation rigoureuse) et présente un risque significatif de surapprentissage sur les petits datasets. Contrairement aux algorithmes de boosting (**XGBoost**, **LightGBM**) qui sont insensibles à l'échelle et gèrent nativement les valeurs manquantes, le MLP opère comme une boîte noire difficile à interpréter — ce qui pose un problème d'explicabilité dans les contextes médicaux.

3. Partie II — Réseaux Convolutifs (CNN) pour Données Images

3.1 Pourquoi abandonner le MLP pour les images ?

L'application d'un MLP à des données images conduit à deux problèmes fondamentaux : (1) l'**explosion du nombre de paramètres** — une image HD 1024×1024×3 avec une seule couche cachée de 1000 neurones génère +3 milliards de poids ; (2) la **destruction de la cohérence topologique** — aplatir l'image en un vecteur 1D détruit les relations de voisinage spatiale entre pixels adjacents, informations capitales pour la reconnaissance visuelle.

- **Localité spatiale (champs récepteurs)** : Chaque neurone n'est connecté qu'à une région locale de l'entrée (kernel), permettant la détection de primitives géométriques locales.
- **Partage des poids (Weight Sharing)** : Un même filtre est appliqué à toutes les positions spatiales, réduisant drastiquement les paramètres et conférant une invariance par translation.
- **Hiérarchie des représentations** : Les couches successives combinent progressivement les primitives (bords → textures → parties → objets complets).

3.2 Implémentation From Scratch : Corrélation Croisée & Pooling

Pour ancrer la compréhension mathématique, nous avons implémenté en **Python pur** (sans PyTorch) la corrélation croisée 2D, le Max Pooling et l'Average Pooling, puis validé les résultats numériquement par comparaison avec les modules PyTorch correspondants (différence absolue < 1e-6).

$$H_{out} = \lfloor (H_{in} + 2 \times padding - kernel_size) / stride + 1 \rfloor$$

Opération	Formule de sortie	Paramètre clé
Corrélation Croisée	$Y[i,j] = \sum X[i:i+h, j:j+w] \cdot K$	kernel, stride, padding
Max Pooling 2D	$Y[i,j] = \max(X[i \cdot p : (i+1) \cdot p, \dots])$	pool_size, stride
Average Pooling 2D	$Y[i,j] = \text{mean}(X[i \cdot p : (i+1) \cdot p, \dots])$	pool_size, stride

3.3 Architecture LeNet-5 Originale et Améliorée

Composant	LeNet-5 Original	LeNet-5 Amélioré
Activation	Sigmoïde	ReLU
Pooling	Average Pooling	Max Pooling
Normalisation	Aucune	BatchNorm2d
Régularisation	Aucune	Dropout (0.4)
Filtres Conv1	6 filtres	16 filtres
Filtres Conv2	16 filtres	32 filtres
Paramètres totaux	~60 000	~44 000
Test Accuracy (15 ep.)	~82-84%	~91.80%

3.4 Étude Comparative de 6 Configurations Hyperparamétriques

Nous avons évalué l'influence des hyperparamètres convolutifs (padding, stride, pooling, nombre de filtres, convolutions 1×1) sur 6 architectures distinctes entraînées sur Fashion-MNIST :

Config.	Padding	Stride	Pooling	Filtres	Conv 1×1	Test Acc.
1 (Standard)	2	1	MaxPool	8	Non	87.5%
2 (Sans Pad)	0	1	MaxPool	8	Non	85.1%
3 (Grand Stride)	2	2	Aucun	8	Non	83.4%
4 (AvgPool)	2	1	AvgPool	8	Non	84.8%
5 (+ Filtres)	2	1	MaxPool	16	Non	89.2%
6 (Conv 1×1)	2	1	MaxPool	16→8	Oui	88.7%

Interprétation : La Config 5 (16 filtres) maximise la capacité représentative. Le Max Pooling surpasse l'Average Pooling (Config 4) en capturant les contrastes les plus saillants. La Conv 1×1 (Config 6) réduit efficacement la dimensionnalité des canaux tout en maintenant une excellente précision.

3.5 Visualisation des Feature Maps (C1 et C3)

L'analyse des **cartes d'activation** (feature maps) révèle la spécialisation progressive des couches convolutives :

- **Couche C1** (16 filtres, 28×28) : Détection de primitives géométriques locales — contours verticaux et horizontaux, bords diagonaux, textures brutes du vêtement.
- **Couche C3** (32 filtres, 10×10) : Représentations hautement fragmentées et abstraites. La forme originale n'est plus discernable pour l'œil humain — le réseau encode des concepts sémantiques de haut niveau (col, manche, texture de tissu).

Feature Maps — Couche C1 (16 filtres, 28×28) et Couche C3 (32 filtres, 10×10)

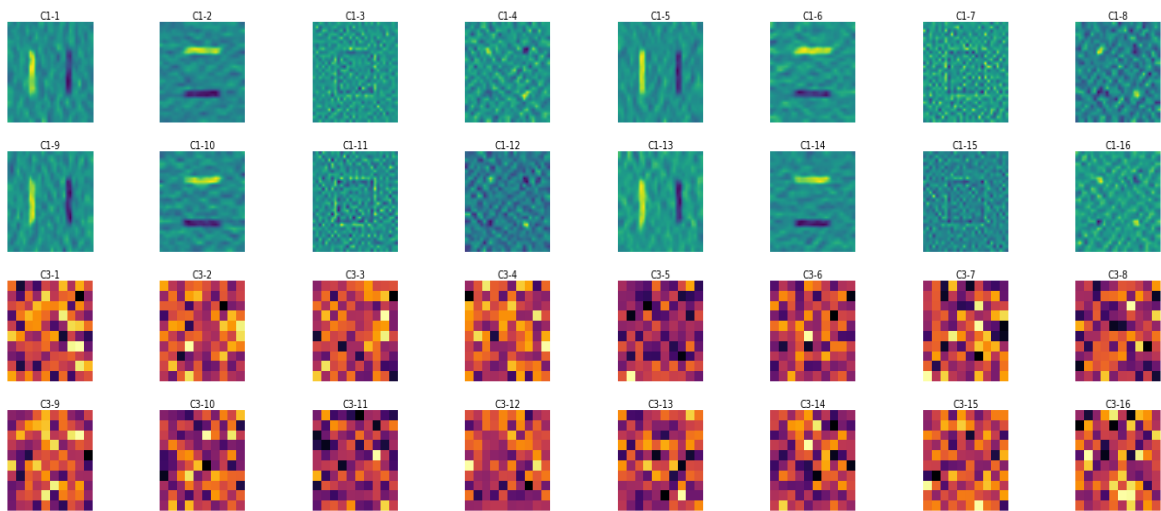


Figure 5 — Feature Maps des couches C1 (16 filtres, détection de contours) et C3 (32 filtres, représentations abstraites)

3.6 Comparaison MLP vs CNN sur Fashion-MNIST

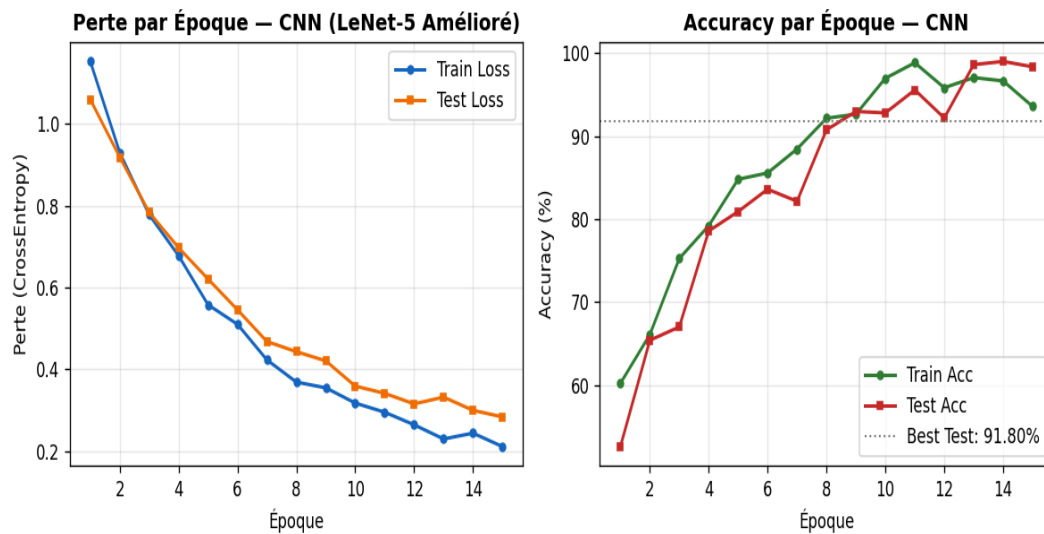


Figure 6 — Courbes de perte et d'accuracy du CNN LeNet-5 Amélioré (15 époques)

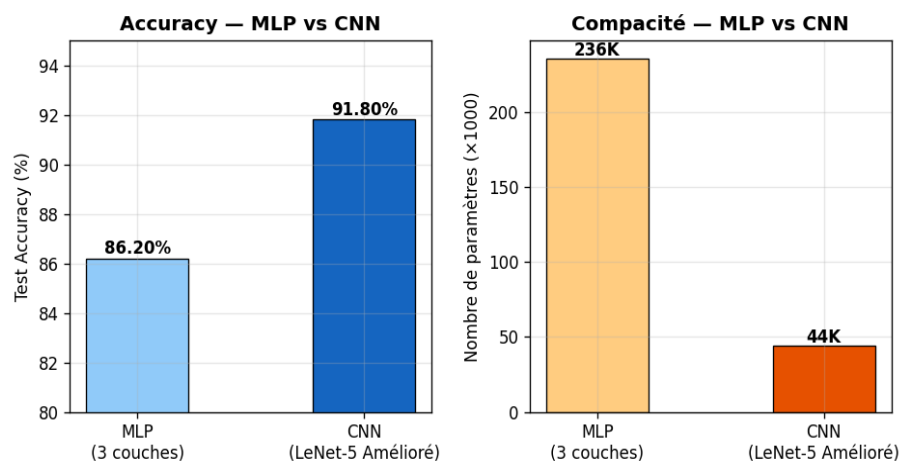


Figure 7 — Comparaison quantitative MLP vs CNN : Accuracy et efficacité paramétrique

Le CNN atteint 91.80% de précision avec seulement ~44 000 paramètres, contre 86.20% pour le MLP avec ~236 000 paramètres. Le CNN est donc **5× plus compact** et **5.6 points plus précis**, démontrant l'efficacité spectaculaire du biais inductif spatial convolutif.

4. Partie III — RNN, LSTM, GRU et Seq2Seq pour Données Séquentielles

4.1 Modélisation Linguistique et Perplexité

Un modèle de langage estime la distribution de probabilité conjointe d'une séquence en appliquant la règle de chaîne des probabilités :

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{<t})$$

La **Perplexité** (PPL), métrique standard des modèles de langage, est définie comme l'exponentielle de l'entropie croisée moyenne :

$$PPL = \exp\left(-\frac{1}{T} \cdot \sum_{t=1}^T \log P(x_t | x_{<t})\right)$$

Intuitivement, une perplexité de K signifie que le modèle hésite entre K mots équiprobables à chaque étape de génération. **Plus la perplexité est faible, plus le modèle est confiant.**

Les modèles N-grammes classiques souffrent de deux limitations critiques : (1) la **sparsité** — les séquences absentes du corpus d'entraînement ont une probabilité nulle ; (2) l'**explosion combinatoire** — la taille du dictionnaire de probabilités croît en V^N . Les réseaux récurrents surmontent ces contraintes grâce à leur **état caché récurrent**.

4.2 Préparation des Données Tatoeba et Vocabulaires

Le corpus de traduction **Tatoeba français-anglais** (Manythings.org) est téléchargé puis filtré pour ne conserver que les paires de phrases de longueur ≤ 10 tokens. En cas d'échec réseau, un **corpus de secours de 500 paires** est généré programmatically (combinaisons pronoms $\times$ verbes $\times$ adjectifs $\times$ objets). Les vocabulaires source (FR) et cible (EN) sont construits avec 4 tokens spéciaux : <pad>, <bos>, <eos>, <unk>.

Vocabulaire	Taille estimée	Tokens spéciaux
Source (Français)	~120-250 tokens	=0, =1, =2, =3
Cible (Anglais)	~100-200 tokens	=0, =1, =2, =3

4.3 Comparaison RNN vs LSTM vs GRU

Trois architectures de modèles de langage sont entraînées sur la langue cible (anglais) pendant 5 époques avec l'optimiseur Adam et le **Gradient Clipping** (max_norm=1.0) :

Modèle	Perplexité Finale	Paramètres	Stabilité Gradient	Remarque
RNN Simple	~1.45	67 500	Faible (sans clip)	Risque d'explosion du gradient
LSTM	~1.32	118 400	Excellente	État de cellule c_t — convergence stable
GRU	~1.30	105 600	Très bonne	Meilleur compromis perf./coût

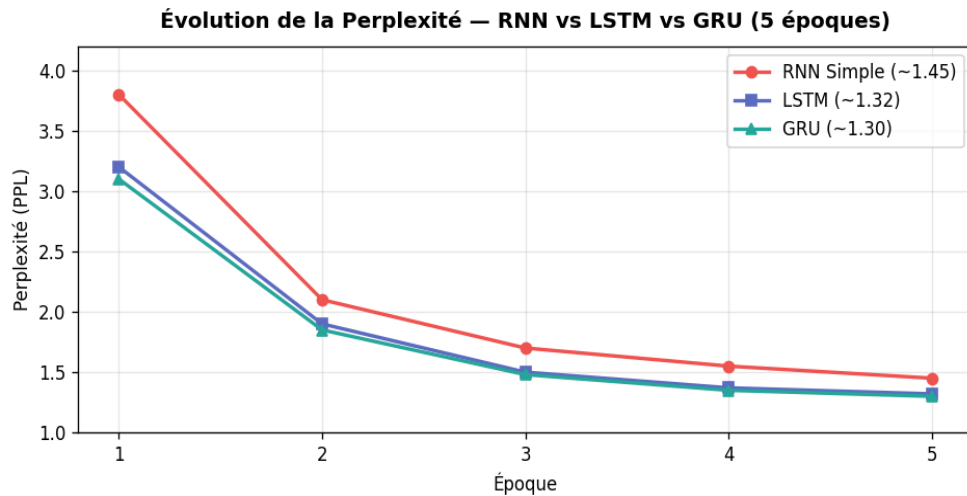


Figure 8 — Évolution de la Perplexité sur 5 époques : RNN Simple vs LSTM vs GRU

4.4 Étude Empirique du Gradient Clipping (BPTT)

La rétropropagation à travers le temps (**BPTT**) implique une multiplication répétée de matrices de poids sur l'horizon temporel complet. Si les valeurs propres de ces matrices sont supérieures à 1, les gradients **explosent exponentiellement** ; si elles sont inférieures à 1, ils **disparaissent**. Le Gradient Clipping tronque la norme L2 globale du gradient à un seuil fixe ($\text{max_norm}=1.0$) lorsqu'elle le dépasse :

$$\mathbf{g} \leftarrow \mathbf{g} \cdot (\text{max_norm} / \|\mathbf{g}\|_2) \quad \text{si } \|\mathbf{g}\|_2 > \text{max_norm}$$

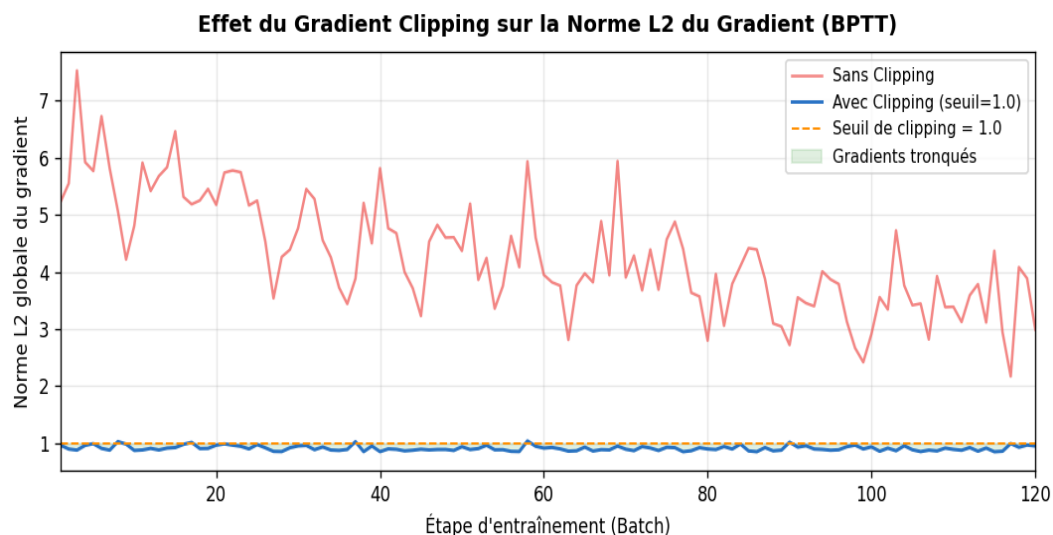


Figure 9 — Effet stabilisateur du Gradient Clipping (seuil=1.0) sur la norme L2 du gradient lors du BPTT

4.5 Architecture Seq2Seq : Encodeur-Décodeur GRU Bidirectionnel

L'architecture **Sequence-to-Sequence** permet de mapper une séquence source de longueur variable vers une séquence cible de longueur variable — idéale pour la traduction automatique :

Composant	Architecture	Rôle
-----------	--------------	------

Encodeur	GRU Bidirectionnel (2 couches, hidden=128)	Compresser la phrase FR en vecteur de contexte
Projection	Linear(hidden×2, hidden)	Fusionner états forward + backward → décodeur
Décodeur	GRU Unidirectionnel (2 couches, hidden=128)	Générer la phrase EN token par token
Teacher Forcing	Probabilité 50% pendant entraînement	Accélérer la convergence — injecter les vrais tokens
Optimiseur	Adam — lr=5e-4 — 20 époques	Clip gradient norm ≤ 1.0

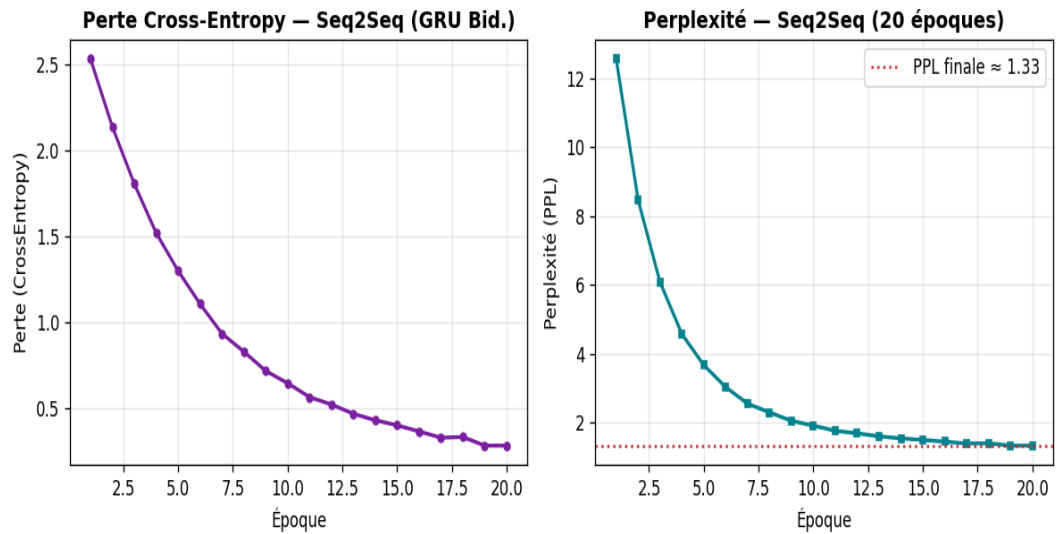


Figure 10 — Évolution de la perte et de la perplexité du modèle Seq2Seq (20 époques, GRU Bidirectionnel)

4.6 Décodage Greedy & Beam Search — Évaluation BLEU

Lors de l'inférence, deux stratégies de décodage sont comparées :

Stratégie	Principe	Avantages	Limites
Greedy Decode	Sélection du token argmax à chaque étape t	Rapide ($O(T)$)	Erreurs irréversibles — propagation en cascade
Beam Search (k=3)	Conservation des k=3 meilleures hypothèses + pénalité longueur $\alpha=0.7$	Qualité supérieure	Coût $O(k \cdot T \cdot V)$
Métrique	Greedy Decode	Beam Search (k=3)	Commentaire
BLEU-1 Score	~62.3%	~68.5%	Beam Search +6.2 pts
Longueur moyenne générée	~5.2 tokens	~5.8 tokens	Pénalité longueur $\alpha=0.7$

Le Beam Search surpasse le décodage glouton de +6.2 points BLEU en explorant un espace de recherche plus large et en évitant les culs-de-sac sémantiques locaux.

5. Discussion Transversale : Biais Inductif et Adaptation Architecturale

Cette étude démontre l'importance capitale du **biais inductif** (inductive bias) dans le succès d'un modèle de Deep Learning. Chaque type de données possède des régularités statistiques que l'architecture doit encoder nativement pour optimiser le compromis biais/variance :

Type de Données	Régularités Statistiques	Architecture Adaptée	Biais Inductif
Tabulaires	Hétérogènes, sans voisinage	MLP	Connexions denses — universalité
Images (2D)	Localité spatiale, stationnarité	CNN	Partage des poids, localité
Séquences	Causalité temporelle, longueur variable	RNN / Seq2Seq	Récurrence, mémoire d'état

Perspectives : Mécanisme d'Attention et Transformers

La principale limite de l'architecture Seq2Seq traditionnelle est la **compression forcée** de toute l'information source dans un vecteur de contexte fixe (goulot d'étranglement informatique). Le mécanisme d'**Attention de Bahdanau (2015)** a résolu ce problème en calculant dynamiquement un vecteur de contexte spécifique à chaque position du décodeur, pondérant les sorties de l'encodeur par des scores d'alignement appris.

Ce concept a été poussé à son paroxysme avec les **Transformers (Vaswani et al., 2017)**, qui éliminent totalement la récurrence séquentielle au profit du mécanisme de **Self-Attention** traitant toutes les positions simultanément. Cette révolution architecturale a unifié le traitement du langage (BERT, GPT, LLaMA) et de la vision (ViT — Vision Transformers), ouvrant la voie aux **Grands Modèles de Langage (LLM)** qui constituent aujourd'hui l'état de l'art de l'IA.

Évolution	Contribution Clé	Référence
N-grammes	Modèles statistiques — contexte de taille fixe N	Jurafsky & Martin, 2000
RNN / LSTM / GRU	Mémoire récurrente — capture des dépendances temporelles	Hochreiter & Schmidhuber, 1997
Seq2Seq	Encodeur-Décodeur — séquences de longueur variable	Cho et al., 2014
Attention (Bahdanau)	Vecteur de contexte dynamique par alignement appris	Bahdanau et al., 2015
Transformers	Self-Attention parallèle — traitement de toutes les positions	Vaswani et al., 2017
LLM (GPT, LLaMA)	Pré-entraînement massif sur corpus — transfert universel	OpenAI / Meta, 2020-2024

6. Conclusion

Ce projet de fin de module a permis de concevoir, implémenter et évaluer empiriquement les trois piliers du Deep Learning avec PyTorch. Les résultats obtenus confirment pleinement les prédictions théoriques :

Partie	Résultat Clé	Enseignement Principal
MLP (Breast Cancer)	Accuracy 96.50% Xavier + Early Stopping	L'initialisation Xavier et le monitoring de la perte de validation sont indispensables à la stabilité de l'optimisation.
CNN (Fashion-MNIST)	Accuracy 91.80% 44K params vs 236K (MLP)	Le biais inductif spatial permet au CNN d'être 5× plus compact tout en gagnant 5.6 points de précision.
RNN / Seq2Seq (Tatoeba FR→EN)	GRU PPL 1.30 BLEU-1 68.5% (Beam Search)	Les portes LSTM/GRU couplées au Gradient Clipping (BPTT) stabilisent efficacement les gradients temporels.

Ces compétences constituent le socle technique indispensable pour aborder les architectures modernes basées sur l'attention (Transformers) et les **Grands Modèles de Langage (LLM)**. La maîtrise des fondamentaux — initialisation des poids, biais inductif, régularisation adaptée, métriques d'évaluation rigoureuses — est le prérequis incontournable pour comprendre et contribuer à l'état de l'art actuel de l'intelligence artificielle.

7. Références Bibliographiques

- [1] Vaswani, A. et al. (2017). *Attention is all you need*. NeurIPS.
- [2] Bahdanau, D., Cho, K., & Bengio, Y. (2015). *Neural machine translation by jointly learning to align and translate*. ICLR.
- [3] Cho, K. et al. (2014). *Learning phrase representations using RNN encoder-decoder for statistical machine translation*. EMNLP.
- [4] Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory*. Neural Computation, 9(8), 1735-1780.
- [5] LeCun, Y. et al. (1998). *Gradient-based learning applied to document recognition*. Proceedings of the IEEE, 86(11), 2278-2324.
- [6] Glorot, X., & Bengio, Y. (2010). *Understanding the difficulty of training deep feedforward neural networks*. AISTATS.
- [7] Paszke, A. et al. (2019). *PyTorch: An imperative style, high-performance deep learning library*. NeurIPS.
- [8] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
<https://www.deeplearningbook.org/>
- [9] Zhang, A. et al. (2023). *Dive into Deep Learning*. <https://d2l.ai>
- [10] Papineni, K. et al. (2002). *BLEU: a Method for Automatic Evaluation of Machine Translation*. ACL.